

SOFTWARE REQUIREMENTS SPECIFICATION

LLM-assisted question generation, practice, coding,
debugging, assessments and placement analytics

Document

SRS Version 1.0

Prepared By

A A Aswathy

Release Label

V1

1. Introduction

1.1 Purpose

The proposed system is an open-source web portal for student placement preparation and skill assessment. It provides structured practice in aptitude, verbal ability, logical reasoning, programming, debugging and computer science fundamentals. It also supports timed assessments, automatic coding evaluation, AI-assisted question generation, performance analytics and placement-readiness indicators.

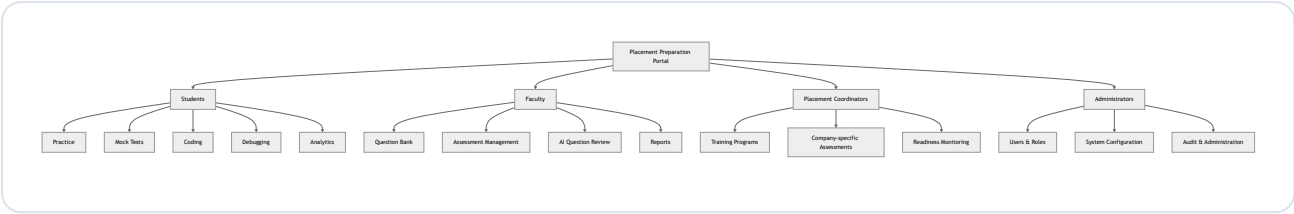
1.2 Vision

Vision: To provide an open-source, intelligent and measurable platform that helps students continuously develop the skills required for technical placements.

1.3 Objectives

- Provide structured placement preparation resources.
- Provide topic-wise practice for aptitude, verbal, coding, debugging and technical subjects.
- Conduct automated mock placement assessments.
- Provide secure online coding evaluation.
- Provide debugging challenges.
- Use LLMs to generate reusable practice questions.
- Validate and review AI-generated content before publication.
- Provide student, batch and department analytics.
- Provide configurable placement-readiness indicators.
- Support self-hosted and multi-college deployment.

2. System Overview



3. Users and Roles

Role	Primary Responsibilities
Student	Practice, attempt tests, solve coding/debugging problems, review results, track progress and recommendations.
Faculty	Create/review questions, create assessments, schedule tests, monitor students and generate reports.
Placement Coordinator	Manage placement preparation programs, company-oriented assessments, readiness monitoring and intervention reports.
Administrator	Manage users, roles, departments, batches, configuration, AI settings and audit logs.

4. Functional Requirements

4.1 Authentication and User Management

- Student, faculty, coordinator and administrator login.
- Registration and profile management.
- Password hashing and secure authentication.
- JWT access token and refresh-token mechanism.
- Role-based access control.
- Session management and logout.
- Optional institutional authentication in future.

4.2 Student Profile

- Name, email, student ID/roll number.
- College/organization, department and batch.
- Skills and preparation preferences.
- Target role or placement track.
- Practice history and progress.

4.3 Question Bank

The question bank shall be the central reusable content repository.

Category	Example Topics
Aptitude	Number systems, percentages, profit & loss, ratio, averages, time & work, probability, permutation & combination, data interpretation.
Logical Reasoning	Series, coding-decoding, blood relations, directions, syllogisms, puzzles, seating arrangements.
Verbal Ability	Vocabulary, grammar, sentence correction, reading comprehension, para jumbles, error detection.
Technical	C, C++, Java, Python, OOP, DBMS, OS, Computer Networks, Data Structures.
Coding	Programming problems with constraints, examples and hidden test cases.
Debugging	Syntax, compilation, logical, runtime and algorithmic bugs.

Each question should support category, topic, subtopic, difficulty, tags, explanation, source, status, version and usage statistics.

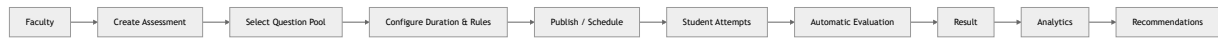
4.4 Practice Engine

- Topic-wise practice.
- Difficulty-based practice.
- Random question selection.
- Immediate or delayed feedback based on mode.
- Explanations and hints.
- Bookmarking.
- Retry incorrect questions.
- Practice history.
- Time and accuracy tracking.

4.5 Assessment and Mock Test Engine

- Create assessments with sections.
- Configure duration, marks and negative marking.
- Schedule assessments.
- Randomize questions and options.

- Configure attempt limits.
- Automatic submission when time expires.
- Section-wise scoring.
- Result generation.



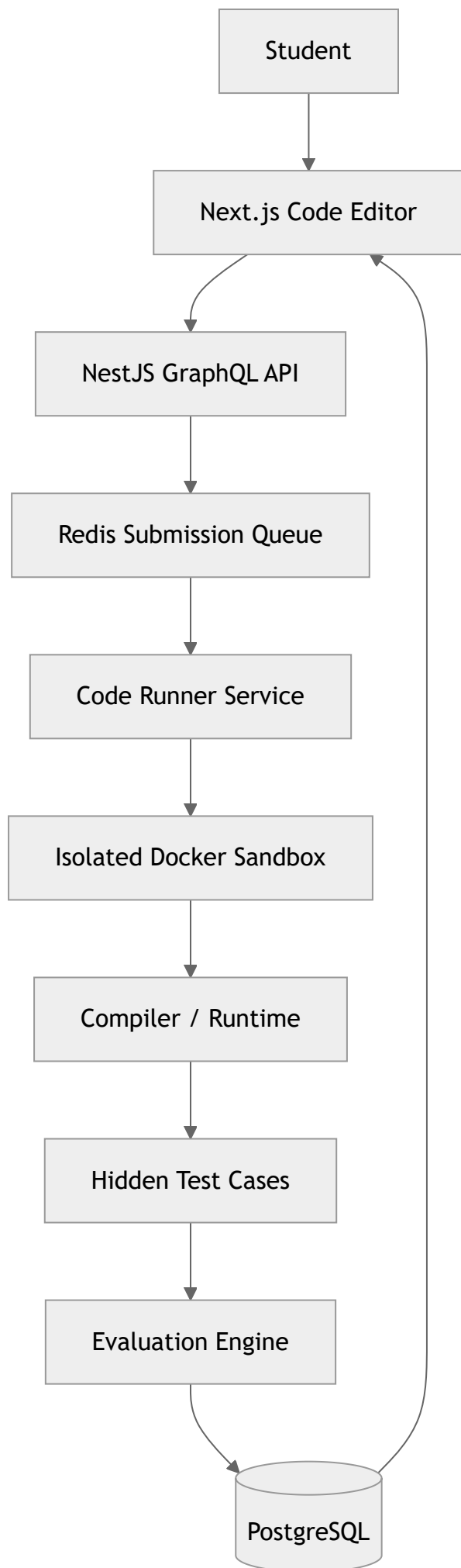
4.6 Coding Module

Students shall solve programming problems through a browser-based code editor.

- Initial languages: C, C++, Java, Python and JavaScript.
- Problem description, input/output format and constraints.
- Sample test cases.
- Hidden test cases.
- Compilation and runtime evaluation.
- Execution time and memory limits.
- Submission history.

4.7 Secure Code Execution

Security requirement: Student code must never execute directly inside the NestJS application process or on the main application host without isolation.



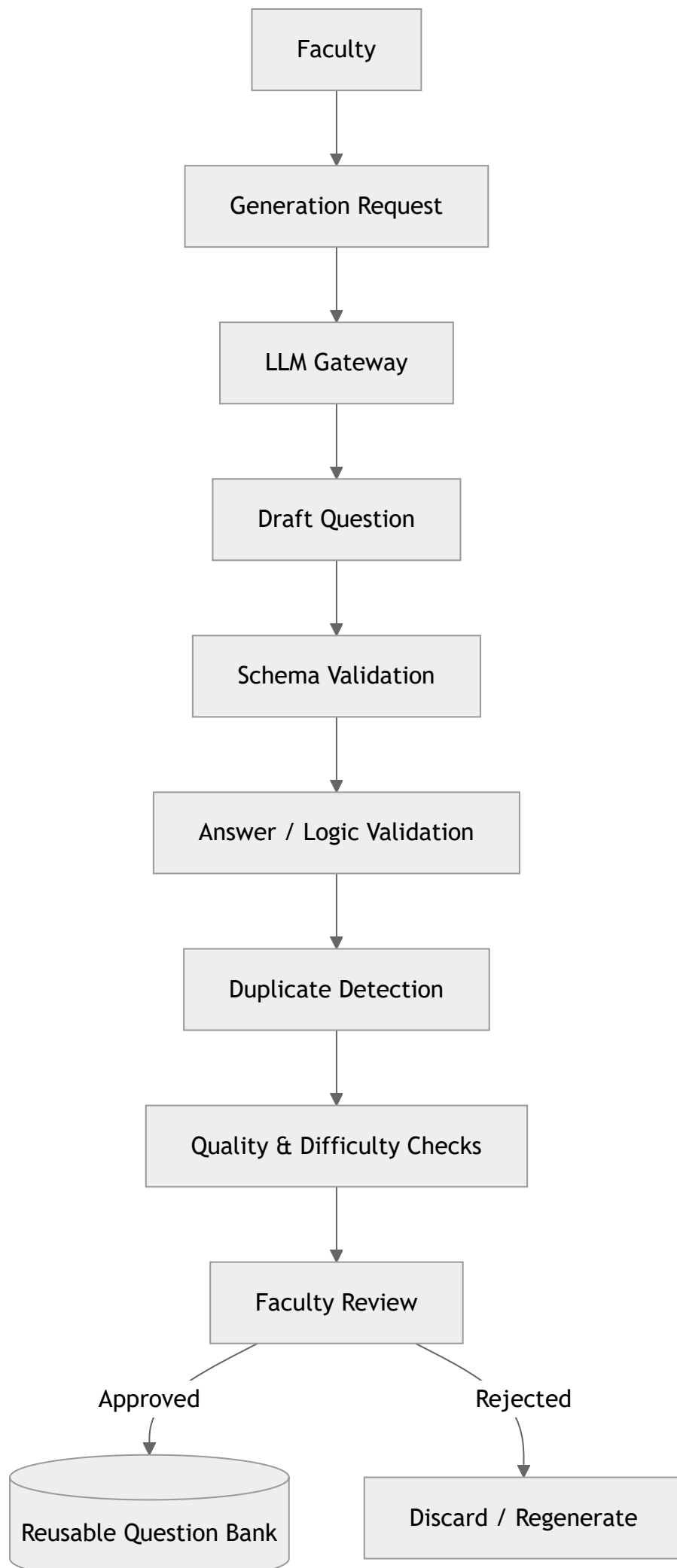
The sandbox should enforce CPU, memory, execution-time, output-size, process and filesystem limits. Network access should normally be disabled.

4.8 Debugging Module

- Provide faulty programs to students.
- Support syntax, compilation, logical, runtime and algorithmic errors.
- Allow code correction and submission.
- Evaluate corrected code against expected behaviour and test cases.
- Provide explanation after submission when appropriate.

4.9 LLM Question Generation

The platform shall provide faculty-controlled AI-assisted question generation. The LLM shall not directly publish content into the live question bank.



LLM Generation Inputs

- Category and topic.
- Number of questions.
- Difficulty distribution.
- Question type.
- Language.
- Target student level.
- Optional placement/company pattern.

LLM Outputs

- MCQ question, options, correct answer and explanation.
- Coding problem, constraints, examples, reference solution and test cases.
- Debugging problem, faulty code, expected behaviour and correction.
- Technical interview questions.

4.10 AI Question Validation

- JSON/schema validation.
- Exactly-one-correct-answer validation for MCQs where applicable.
- Numerical answer verification using deterministic computation where possible.
- Coding reference-solution and test-case execution.
- Duplicate/similarity detection.
- Difficulty consistency checks.
- Faculty approval workflow.

4.11 Question Versioning

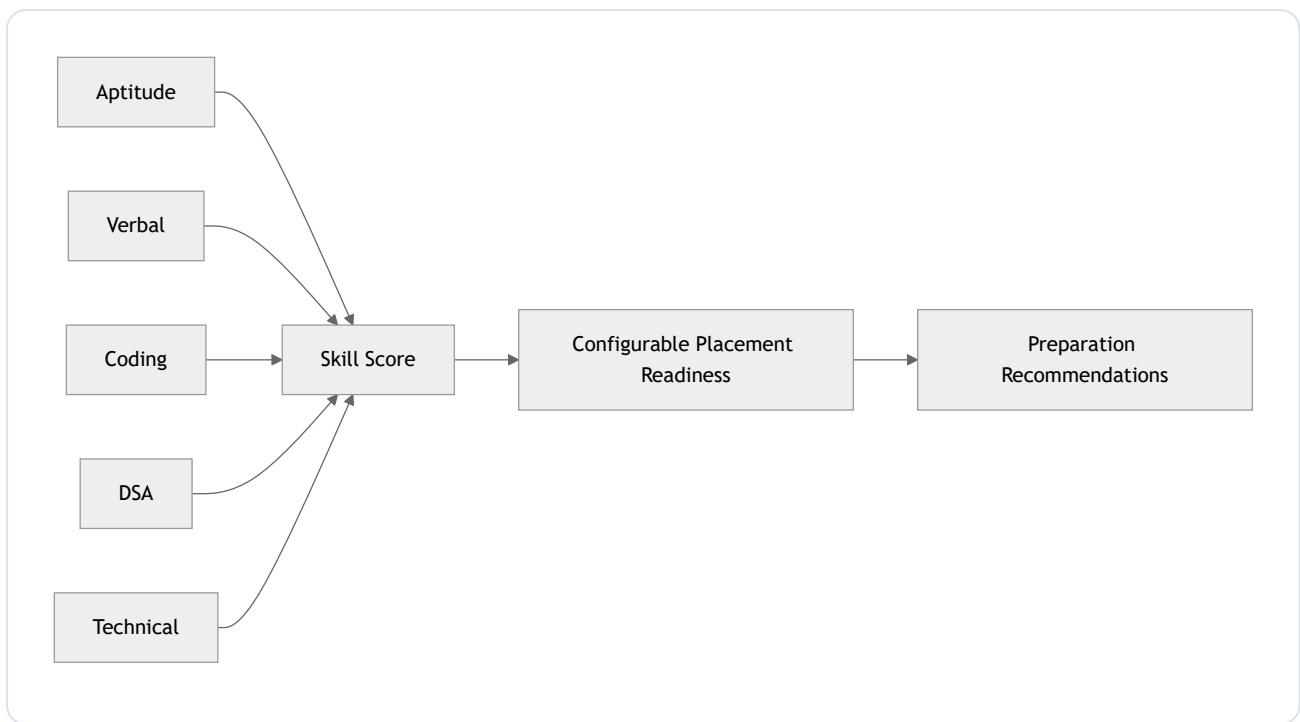
Field	Purpose
Question ID	Permanent identity.
Version	Track edits and corrections.
Generated By	Faculty, admin or AI.
LLM Model	Model used for generation.
Prompt Version	Generation-template version.
Review Status	Draft, pending review, approved, published or retired.
Reviewed By	Faculty/admin reviewer.

4.12 Analytics

- Accuracy.
- Questions attempted and solved.
- Average time per question.
- Topic-level performance.
- Difficulty-level performance.
- Coding acceptance rate.
- Debugging success rate.
- Test-wise performance.
- Progress over time.

4.13 Placement Readiness

The system shall provide a configurable preparation/readiness indicator based on selected skill categories. It should be clearly described as an indicative preparation metric rather than a guaranteed placement prediction.



4.14 Personalized Recommendations

Initially, recommendations should be rule-based and explainable. Example: if Probability accuracy falls below a configured threshold, recommend a Probability practice set and a related assessment.

4.15 Gamification

- Points for solving questions and completing assessments.
- Badges.
- Practice streaks.
- Optional leaderboards by batch, department or assessment.
- Privacy controls for public ranking.

4.16 Faculty and Placement Coordinator Dashboard

- Student count and active users.
- Assessment participation.
- Average scores.
- Weak-topic identification.
- Individual student reports.
- Batch/department comparisons.
- Students requiring intervention.

4.17 Reports

- Student performance report.
- Batch performance report.
- Assessment report.
- Topic-wise skill report.
- Coding performance report.
- Placement-readiness report.
- Export to CSV/Excel/PDF in later phases.

5. Assessment Integrity and Anti-Cheating

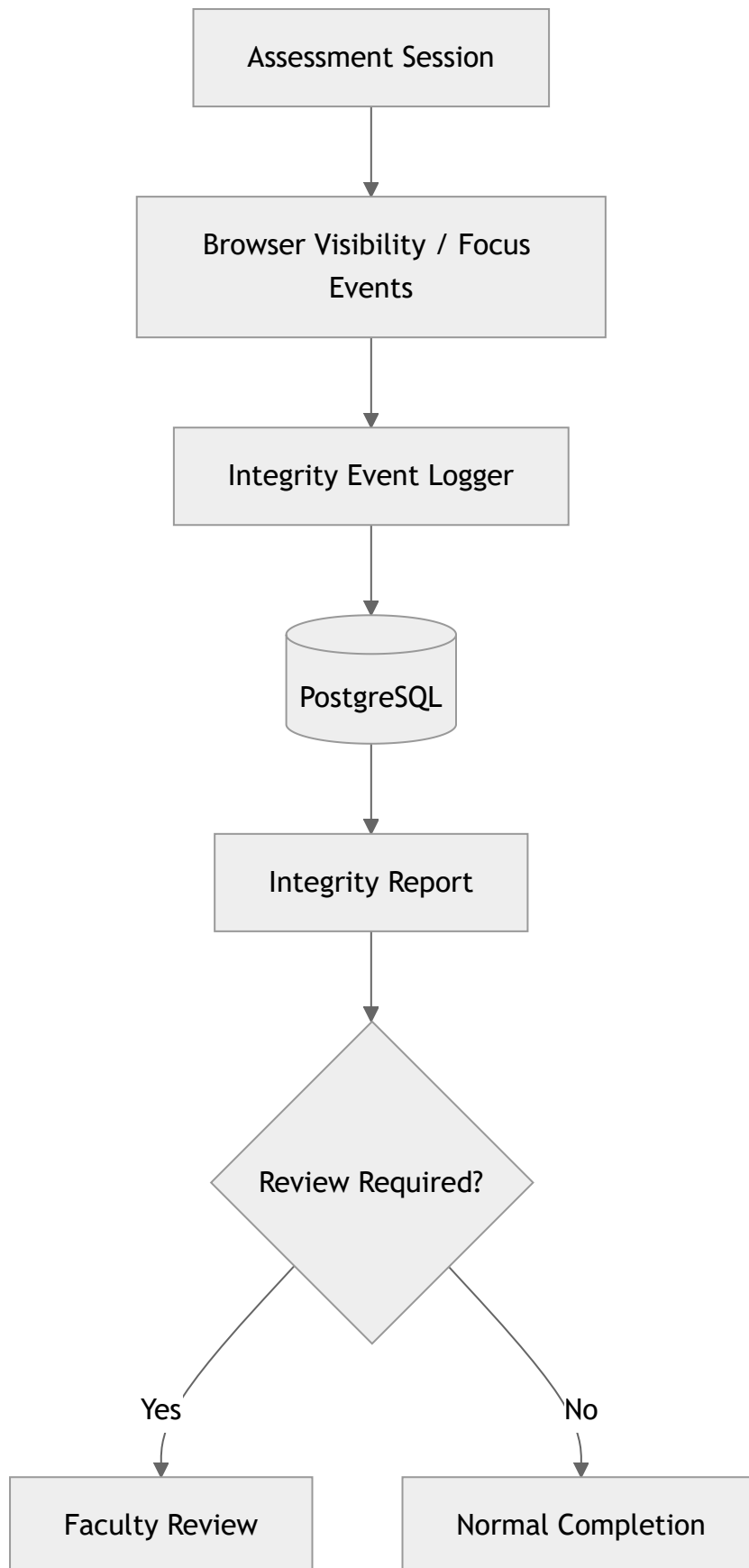
Design principle: The system should detect and record suspicious assessment events rather than claim that a browser can be made completely cheat-proof.

5.1 Basic Integrity Monitoring

- Browser/tab visibility changes.
- Browser focus loss.
- Fullscreen exit.
- Copy/paste events where technically detectable.
- Multiple active sessions.
- Unexpected session interruptions.
- Unusual answer timing patterns.

5.2 Browser Minimization

The platform should not attempt to permanently prevent browser minimization. Browser visibility/focus events can be logged as integrity events. Multiple events may trigger a faculty review flag.



5.3 Proctored Mode — Future

- Fullscreen requirement.

- Optional camera/microphone.
- Identity verification.
- Screen monitoring/recording where institutionally approved.
- Explicit consent and retention policy.

5.4 Privacy Requirements

If webcam, microphone, identity verification or screen recording is introduced, the system must provide clear consent, collection purpose, retention duration, access control and deletion policies.

6. Non-Functional Requirements

6.1 Performance

- Normal application pages should target approximately two seconds or better under normal load.
- Coding execution should be asynchronous.
- Long-running AI generation should use background jobs.

6.2 Scalability

- Support thousands of students.
- Support concurrent assessments.
- Scale code-runner workers independently.
- Scale AI-generation workers independently.
- Support horizontal scaling of stateless application services.

6.3 Availability and Reliability

- Graceful handling of network interruptions.
- Autosave assessment responses where appropriate.
- Retry mechanisms for asynchronous jobs.
- Database backup and recovery strategy.
- Audit logging for important administrative operations.

6.4 Security

- HTTPS in deployment.
- Password hashing.
- JWT/refresh-token security.
- RBAC and authorization checks at resolver/service level.
- Input validation.
- Rate limiting.
- Audit logs.
- Secure secrets management.
- Isolated code execution.

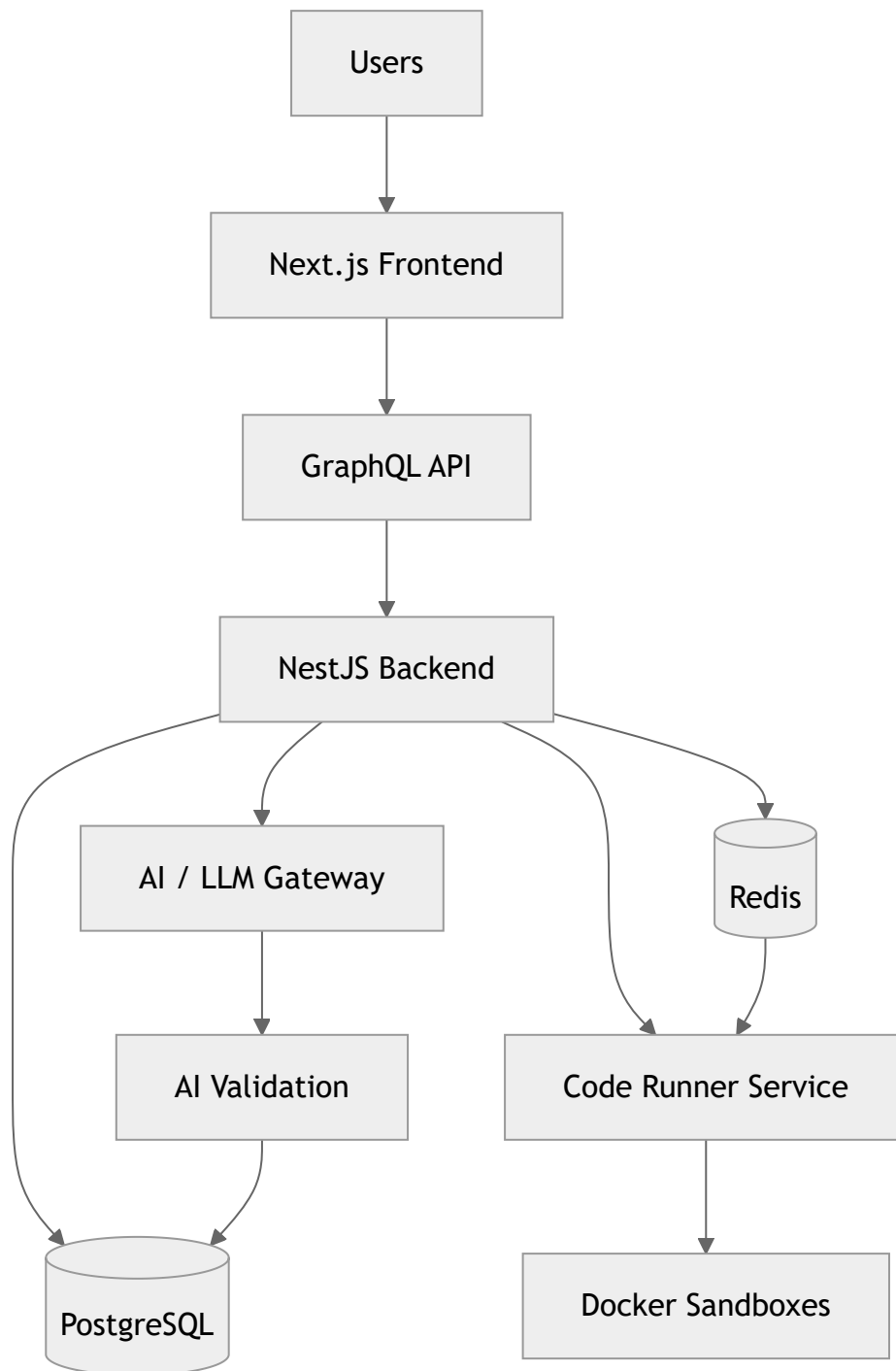
7. Technology Architecture

Component	Recommended Technology
Frontend	Next.js + TypeScript + Tailwind CSS
Backend	NestJS + TypeScript
API	GraphQL
Database	PostgreSQL
ORM	Prisma
Vector Search	pgvector
Cache/Queue	Redis
Code Editor	Monaco Editor or equivalent
Code Execution	Isolated Docker-based runner
AI	LLM Gateway supporting external or local models
Deployment	Docker / Docker Compose
Source Control	Git / GitHub

7.1 Why PostgreSQL?

PostgreSQL is recommended over MongoDB because the platform contains strongly related entities such as students, batches, assessments, attempts, questions, responses and scores. PostgreSQL provides transactions, referential integrity, aggregation and reporting capabilities. PostgreSQL can also store flexible JSONB content and use pgvector for semantic similarity, making it suitable for AI-generated questions without requiring a separate MongoDB database.

8. High-Level Architecture

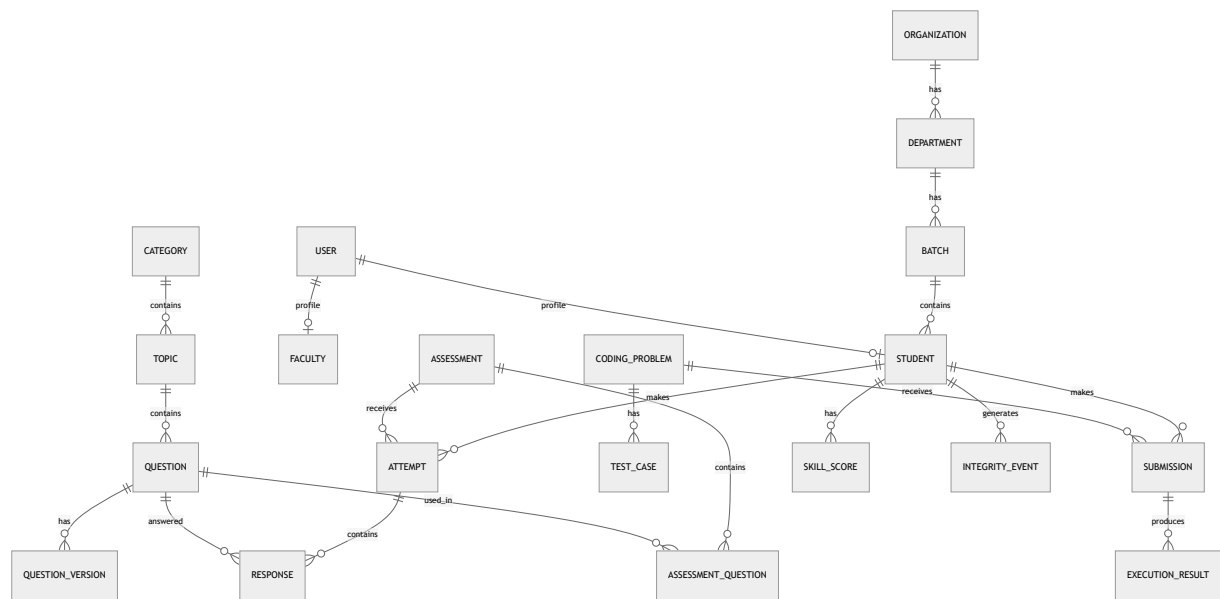


9. Backend Module Structure

```
src/  
├─ auth/  
├─ users/  
├─ students/  
├─ faculty/  
├─ organizations/  
├─ departments/  
├─ batches/  
├─ questions/  
├─ categories/  
├─ assessments/  
├─ attempts/  
├─ coding/  
├─ debugging/  
├─ submissions/  
├─ code-runner/  
├─ ai/  
├─ analytics/  
├─ recommendations/  
├─ leaderboard/  
├─ notifications/  
├─ reports/  
└─ audit/
```

The backend should initially be implemented as a **modular monolith**. The code runner and potentially AI workers can later be separated into independent services when scale requires it.

10. Database Design



10.1 Core Tables

```
users
roles
students
faculty
organizations
departments
batches

categories
topics
questions
question_options
question_tags
question_versions

coding_problems
test_cases
debugging_problems

assessments
assessment_sections
assessment_questions
attempts
```

responses

submissions

execution_results

student_progress

skill_scores

recommendations

badges

student_badges

leaderboards

ai_generation_jobs

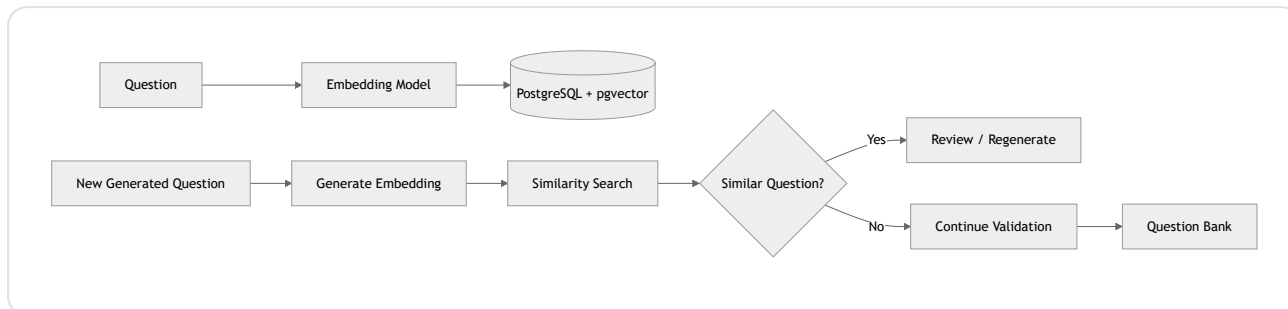
ai_generation_results

integrity_events

notifications

audit_logs

11. AI and Semantic Search Architecture



Embedding-based similarity should be used to detect semantically similar questions and reduce duplicate content generated by the LLM.

12. Open-Source and Deployment Requirements

- Public source repository.
- Open-source license.
- Complete README.
- Developer setup guide.
- Docker Compose deployment.
- Database migration scripts.
- Seed/sample data.
- Environment variable documentation.
- API documentation.
- Contribution guidelines.
- Issue and feature-request templates.

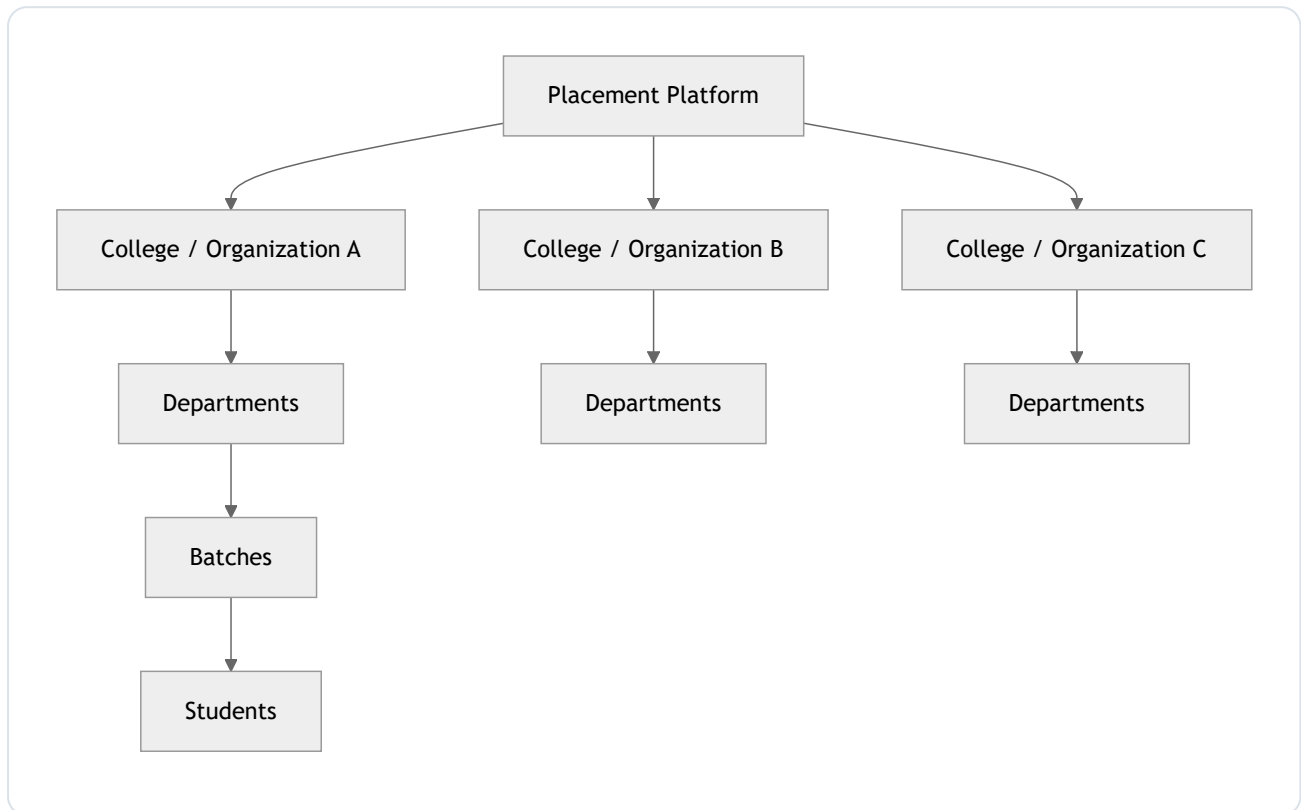
12.1 Target Deployment

```
git clone <repository>
cp .env.example .env
docker compose up -d
```

The deployment should bring up the web application, API, PostgreSQL, Redis and required worker services.

13. Multi-College Architecture

The platform should be designed for multi-tenant or self-hosted college deployment from the beginning.



Organization, department and batch should therefore be first-class entities rather than hard-coded college-specific fields.

14. Development Phases

Phase 1 — Platform Foundation CRITICAL

- Next.js frontend setup.
- NestJS backend setup.
- GraphQL configuration.
- PostgreSQL + Prisma.
- Docker Compose.
- User registration/login.
- JWT and refresh tokens.
- RBAC.
- Organization, department and batch structure.
- Basic student/faculty/admin dashboards.

Deliverable: Users can securely log in and access role-specific dashboards.

Phase 2 — Question Bank & Practice CRITICAL

- Question categories and topics.
- MCQ question management.
- Aptitude, verbal, reasoning and technical questions.
- Difficulty levels.
- Practice engine.
- Random questions.
- Answer explanations.
- Bookmarks.
- Practice history.

Deliverable: Students can practice a reusable question bank.

Phase 3 — Assessment Engine

CRITICAL

- Assessment creation.
- Sections and question pools.
- Timer.
- Randomization.
- Negative marking.
- Scheduling.
- Attempt limits.
- Automatic submission.
- Result generation.

Deliverable: Faculty can conduct placement-style mock tests.

Phase 4 — Coding & Debugging

CRITICAL

- Monaco-based coding interface.
- C/C++/Java/Python/JavaScript.
- Problem management.
- Test-case management.
- Submission queue.
- Docker sandbox.
- Compilation/runtime evaluation.
- Debugging challenges.
- Submission history.

Deliverable: Students can safely submit and evaluate real programs.

Phase 5 — LLM Question Generation

CRITICAL

- LLM gateway.
- Prompt templates.
- Structured JSON generation.
- MCQ generation.
- Coding problem generation.
- Debugging problem generation.
- Automated validation.
- Duplicate detection.
- Question versioning.
- Faculty approval workflow.
- AI generation history.

Deliverable: Faculty can generate, validate, review and save reusable AI-assisted questions.

Phase 6 — Analytics & Placement Readiness

HIGH

- Student analytics dashboard.
- Topic-level performance.
- Time analysis.
- Coding performance.
- Batch analytics.
- Skill scores.
- Configurable placement-readiness score.
- Faculty reports.

Deliverable: Faculty can identify skill gaps and students can understand their preparation level.

Phase 7 — Recommendations & Gamification

MEDIUM

- Rule-based recommendations.
- Weak-topic detection.
- Personalized practice sets.
- Points.
- Badges.
- Streaks.
- Leaderboards.

Deliverable: Students receive targeted preparation and motivation features.

Phase 8 — Assessment Integrity & Proctoring

HIGH

- Focus/visibility event logging.
- Fullscreen monitoring.
- Copy/paste event logging where technically available.
- Multiple-session detection.
- Integrity reports.
- Faculty review workflow.
- Optional proctored mode.
- Privacy/consent controls.

Deliverable: The platform can support institution-configurable assessment integrity policies.

Phase 9 — Multi-College Platform

- Multi-tenancy improvements.
- Organization administration.
- Institution-specific configurations.
- Institution-level question banks.
- Global reusable question repository.
- Institution-specific reports.

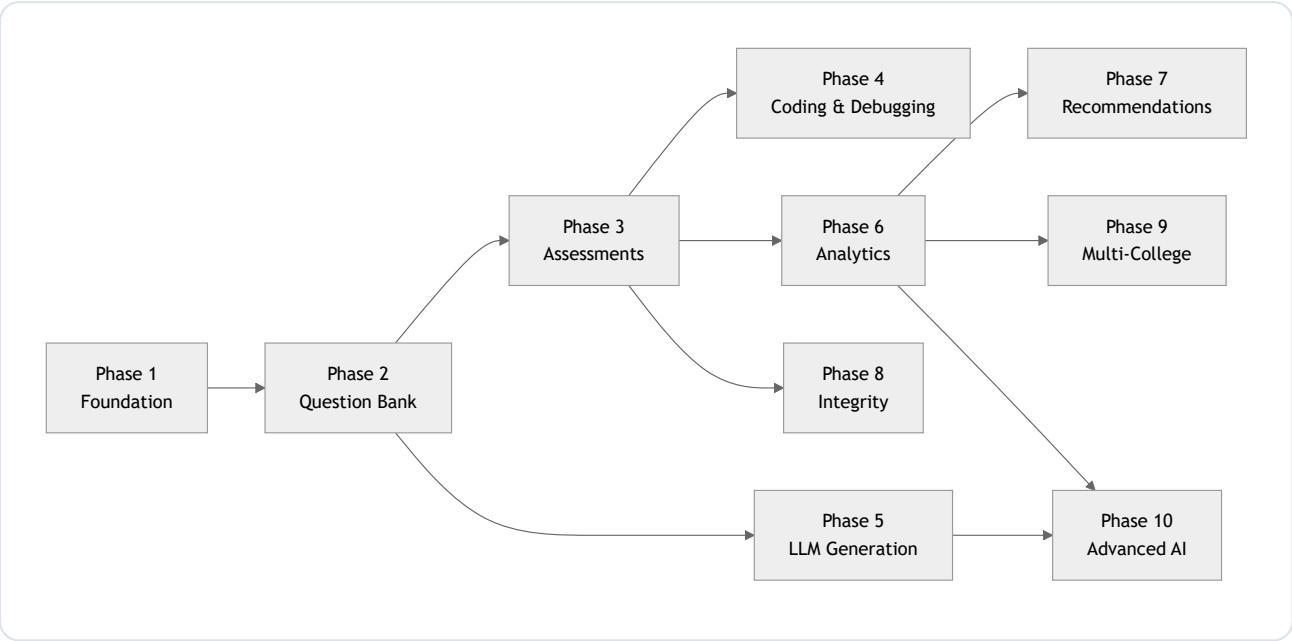
Deliverable: Independent colleges can deploy and administer the platform.

Phase 10 — Advanced AI & Research Features

- Adaptive testing.
- AI tutor.
- AI code review.
- Advanced question difficulty estimation.
- Learning-path recommendation.
- Interview simulator.
- Research-oriented analytics.

Deliverable: The platform evolves into an intelligent placement preparation system.

15. Phase Dependency



16. MVP Definition

Recommended first public release: Phase 1 + Phase 2 + Phase 3 + Phase 4. This creates a useful placement platform before advanced AI features are introduced.

MVP Feature	Included?
Authentication/RBAC	Yes
Student dashboard	Yes
Faculty dashboard	Yes
Aptitude practice	Yes
Verbal practice	Yes
Technical MCQs	Yes
Mock tests	Yes
Coding judge	Yes
Debugging	Basic/Phase 4
LLM generation	Phase 5
Advanced analytics	Phase 6
Proctoring	Phase 8

17. Key Design Decisions

1. **PostgreSQL over MongoDB:** relational assessment data, reporting and transactions are central to the platform; JSONB provides flexibility for AI content.
2. **pgvector:** use PostgreSQL vector search for semantic duplicate detection before introducing another database.
3. **Modular monolith first:** keep NestJS modules together initially; separate code execution and AI workers when scale requires it.
4. **AI does not auto-publish:** generated questions must pass validation and faculty review before becoming reusable published content.
5. **Secure code runner:** execute untrusted code only inside isolated sandboxes.
6. **Integrity events rather than browser locking:** browser minimization/tab switching should be treated as signals for review, not as a claim of cheat prevention.
7. **Multi-college ready:** organization, department and batch must be designed into the data model from Phase 1.

18. Future Enhancements

- Company-specific preparation tracks.
- Resume analysis.
- AI interview preparation.
- Technical interview simulator.
- Adaptive assessments.
- AI tutor.
- AI code review.
- Advanced proctoring.
- Mobile/PWA support.
- Institutional SSO.
- Research dashboards and learning analytics.

19. Conclusion

The proposed platform is intended to be more than an online question-practice website. Its architecture combines a reusable question bank, assessment engine, secure coding judge, debugging environment, LLM-assisted content generation, analytics and placement-readiness support.

The recommended technology stack—**Next.js, NestJS, GraphQL, PostgreSQL, Prisma, Redis, Docker and an LLM gateway**—provides a strong foundation for an open-source system that can begin as a college placement-support platform and later evolve into a multi-college or research-oriented intelligent learning platform.

Open-Source Placement Preparation & Skill Assessment Portal

SRS Version 1.0 · Prepared by A A Aswathy · Release Label: A A Aswathy V1